Лабораторная работа №5

Функциональные возможности Python: функции высшего порядка, итераторы и генераторы

1. Цель работы

Получить практические навыки применения функций высшего порядка, lambda, map, filter, reduce, сортировки и генераторов.

2. Постановка индивидуального задания

Вариант 8. Заказы. Для заказов хранятся номер, клиент, стоимость и статус. Требуется фильтровать активные заказы, получать номера, сортировать по стоимости, вычислять общую стоимость, проверять наличие отменённых заказов и реализовать генератор заказов выбранного клиента.

3. Краткие теоретические сведения

map() преобразует элементы, filter() отбирает подходящие, reduce() сворачивает последовательность в одно значение, sorted() сортирует по ключу, any() проверяет наличие хотя бы одного совпадения. yield создаёт генератор, который выдаёт результаты лениво.

4. Исходный код программы

Файл main.py

```
from functools import reduce

orders = [
    {"number": 101, "customer": "Иванов", "cost": 1500.0, "status": "новый"},
    {"number": 102, "customer": "Петров", "cost": 750.0, "status": "отменён"},
    {"number": 103, "customer": "Иванов", "cost": 3200.0, "status": "оплачен"},
    {"number": 104, "customer": "Сидоров", "cost": 2100.0, "status": "отправлен"},
]

def active_orders(items):
    return list(filter(lambda order: order["status"] != "отменён", items))
```

```
def order_numbers(items):
    return list(map(lambda order: order["number"], items))

def sort_by_cost(items):
    return sorted(items, key=lambda order: order["cost"])

def total_cost(items):
    return reduce(lambda total, order: total + order["cost"], items, 0.0)

def has_cancelled(items):
    return any(order["status"] == "отменён" for order in items)

def orders_for_customer(items, customer):
    """Лениво возвращает заказы выбранного клиента."""
    for order in items:
        if order["customer"].casefold() == customer.casefold():
            yield order

def main():
    print("Активные заказы:", active_orders(orders))
    print("Номера заказов:", order_numbers(orders))
    print("Сортировка по стоимости:", sort_by_cost(orders))
    print(f"Общая стоимость: {total_cost(orders):.2f} руб.")
    print("Есть отменённые заказы:", "да" if has_cancelled(orders) else "нет")
    customer = input("Введите клиента для генератора (Иванов/Петров/Сидоров): ").strip()
    print("Заказы клиента:")
    for order in orders_for_customer(orders, customer): print(order)

if __name__ == "__main__": main()
```

5. Объяснение работы программы

Функции active_orders(), order_numbers(), sort_by_cost(), total_cost() и has_cancelled() демонстрируют операции над коллекцией. orders_for_customer() использует yield и не создаёт отдельный список всех заказов клиента.

6. Примеры работы программы в виде снимков экрана

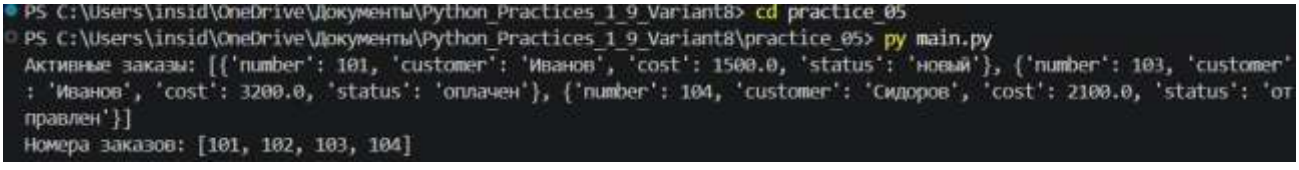


Рисунок 5.1 – Результат фильтрации активных заказов и получение номеров.

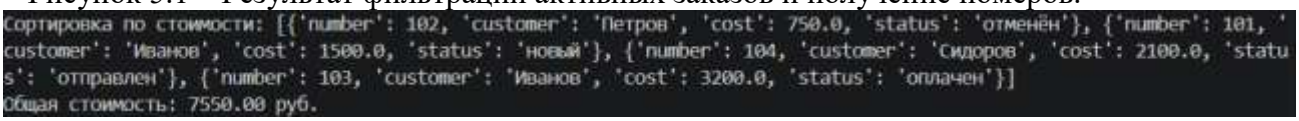


Рисунок 5.2 – Сортировка заказов и расчёт общей стоимости.

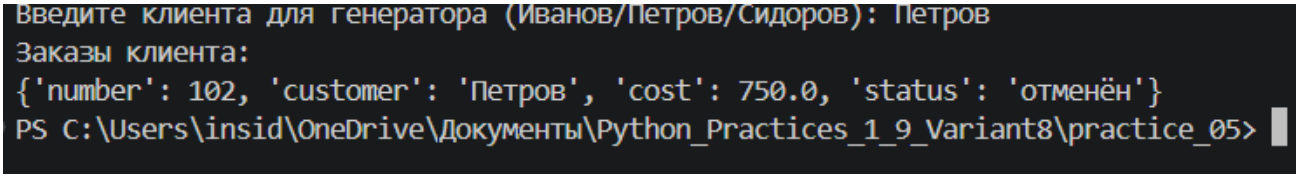


Рисунок 5.3 – Работа генератора для выбранного клиента.

7. Вывод

В ходе выполнения лабораторной работы были реализованы требования индивидуального задания варианта 8. Функции active_orders(), order_numbers(), sort_by_cost(), total_cost() и has_cancelled() демонстрируют операции над коллекцией. orders_for_customer() использует yield и не создаёт отдельный список всех заказов клиента. Программа разделена на логические компоненты, содержит проверку типичных ошибок и сопровождается автоматическими тестами.

Выполнил: Куршаков Егор Игоревич